CÉGEP De Chicoutimi

LIVRABLE 5 : Bilan

PARTAGE DE DÉPENSES - Dév. App. Avancé | 420-D60-CH | Gr. 01

Henri Pitre & Tommy-Frédérique Bernaquez
2026-03-27

TABLE DES MATIÈRES

TABLE DES MATIÈRES 2
1. PROBLÈMES RENCONTRÉS 3
1.1 Clarification besoins du client 3
1.2 Travail en double 3
1.3 Problème GitHub 3
1.4 Problème de communication 4
1.5 Planification en cours de route 4
1.6 Cas d'utilisation manquant 4
2. FACTEURS CLÉS 5
2.1 La compréhension du besoin client 5
2.2 L'utilisation de User Control 5
2.3 L'utilisation de style 5
3. AMÉLIORATION CONTINUE 6
3.1 Planification plus exhaustive 6
3.2 Cas d'utilisation plus exhaustif 6
3.3 Meilleure communication 6
4. BONS COUPS 7
4.1 Henri 7
4.2 Tommy-Frédéric 7

1. PROBLÈMES RENCONTRÉS

1.1 Clarification besoins du client

Au départ, la compréhension du projet orientait l'équipe vers la conception d'une application simple de calcul de remboursements. Or, le client souhaitait une expérience complète de gestion d'activités et de dépenses, incluant la gestion des droits par rôles, la persistance des données, et une interface riche avec plusieurs vues fonctionnelles. Ce besoin n'ayant pas été formellement documenté dès le départ, l'équipe a dû revoir et élargir la portée du projet en cours de route. La leçon retenue est qu'il est essentiel de valider explicitement avec le client l'étendue complète de ses attentes avant d'entamer toute phase de développement, même lorsque le besoin semble évident à première vue.

1.2 Travail en double

À plusieurs reprises, les deux membres de l'équipe ont travaillé sur les mêmes composants sans coordination suffisante, ce qui a engendré des doublons dans le code et des conflits à la fusion. Ce problème a allongé inutilement certaines phases du projet et a nécessité des révisions supplémentaires pour harmoniser le travail accompli.

1.3 Problème GitHub

Des *merge conflicts* sont survenus sur le dépôt GitHub, principalement en raison d'un manque de convention sur les branches de travail. L'absence d'une stratégie de branchement claire dès le départ est la cause principale de ce problème.

1.4 Problème de communication

La communication entre les deux coéquipiers n'a pas toujours été optimale. Certaines décisions de conception ont été prises individuellement sans en aviser l'autre, ce qui a parfois créé des incohérences dans l'application et obligé à revenir sur des choix déjà implémentés. Une communication plus régulière et structurée aurait permis d'éviter plusieurs de ces situations.

1.5 Planification en cours de route

La planification des tâches a été effectuée de façon réactive plutôt que proactive. Certaines fonctionnalités ont été ajoutées ou redéfinies en cours de route, ce qui a perturbé l'ordre logique du développement. Cette approche improvisée a nui à l'efficacité de l'équipe et a rendu la gestion des priorités plus difficile.

1.6 Cas d’utilisation manquant

Plusieurs cas d'utilisation n'ont pas été documentés dans le cahier des charges, notamment les scénarios de gestion des erreurs et certains flux de navigation secondaires. Ce manque a conduit à des décisions de conception improvisées lors du développement, sans cadre de référence clair pour valider le comportement attendu.

2. FACTEURS CLÉS

2.1 La compréhension du besoin client

Le client souhaitait une application offrant une expérience complète de gestion d'activités et de dépenses, et non pas simplement un outil de calcul de remboursements. Cette exigence a fortement orienté les choix architecturaux, la gestion des droits par rôles, ainsi que la conception d'une interface riche avec plusieurs vues fonctionnelles.

2.2 L’utilisation de User Control

Le recours aux User Controls WPF a été un facteur clé de succès dans la conception de l'interface. Cette approche a permis de créer des composants réutilisables, cohérents visuellement et plus faciles à maintenir. En particulier, le composant *DualListView* a grandement simplifié l'interface de création de dépenses et d'assignation de participants, qui est l'une des fonctionnalités les plus complexes de l'application.

2.3 L’utilisation de style

L'utilisation de styles globaux WPF (ressources de style partagées) a assuré une cohérence visuelle à travers toutes les vues de l'application. Cela a également accéléré le développement de nouvelles interfaces, puisque les couleurs, typographies et comportements visuels étaient centralisés et réutilisables. Cette décision a contribué directement à la qualité perçue de l'application.

3. AMÉLIORATION CONTINUE

3.1 Planification plus exhaustive

Pour de futurs projets, il serait bénéfique d'investir davantage de temps dans la phase de planification initiale. L'établissement d'un calendrier détaillé avec des jalons clairs, une répartition précise des tâches et une identification préalable des dépendances entre composants permettrait d'éviter les blocages et le travail en double. Des outils comme un tableau Kanban ou un diagramme de Gantt pourraient être utilisés dès le départ.

3.2 Cas d'utilisation plus exhaustif

La rédaction de cas d'utilisation plus complets, incluant les scénarios alternatifs et les cas d'erreur, constitue une amélioration majeure à apporter. Des *use cases* bien définis servent de référence tout au long du développement et facilitent également la conception des tests. Dans un prochain projet, chaque fonctionnalité devrait être accompagnée d'au moins un scénario principal et un scénario d'exception.

3.3 Meilleure communication

L'adoption de pratiques de communication structurées et une convention de branchement Git clairement établie dès le début, permettrait de réduire significativement les conflits et les malentendus. Une meilleure communication contribuerait également à maintenir une vision commune du projet tout au long de sa réalisation.

4. BONS COUPS

4.1 Henri

4.1.1 Service de navigation dynamique et facile à associer

Le INavigationService et sa classe qui en découle permet de gérer la navigation visuelle et en même temps le passage des données entre chaque vue de façon uniforme. Cela garantit l'application du MVVM, de la réutilisation, et d'une façon de faire facile à comprendre et qui ne change pas de vue en vue. Cela a également permis d'alléger les vues et les vue-modèles de paramètres de passages et de méthodes répétitives.

4.1.2 Optimisation de l'expérience utilisateur

Dans le but de rendre l'application plus intuitive et désirable, un œil particulier a été jeté sur l'efficacité de toutes les composantes faisant partie du projet. Ainsi, l'ajout de couleurs, d'icônes +/-, d'émoticônes simplistes telles que la carte bancaire ou la couronne pour désigner un participant propriétaire ou encore l'ajout de texte en survol (tooltip) a su rendre l'application plus agréable, compréhensible et moins monotone.

4.1.3 Résumé efficace des données en un endroit

En résumant toutes les données en endroits précis et logiques, on gagne grandement en réflexion et en temps, pour les développeurs tout comme les utilisateurs. Ainsi, l'affichage de détails d'une dépense en une seule ligne dans un listview avec toutes les informations disponibles en un endroit est bien plus pratique et facile à implémenter.

4.2 Tommy-Frédéric

4.2.1 Magnifiques animations

Ce détail, appliqué notamment sur des boutons ou encore sur l'icône d'œil, témoigne d'un souci du détail et d'une volonté de livrer une expérience utilisateur agréable et mémorable, puisque son utilité n'est pas fonctionnelle mais plutôt esthétique.

4.2.2 Centralisation des styles

La centralisation des styles des éléments XAML dans un fichier « GlobalStyles.xaml » a su nous sauver bon nombre de temps en évitant une énorme répétition au sein de toutes les vues de l'application. Ainsi, les modifications visuelles relatives à un élément seul se faisaient en un seul endroit et se synchronisaient partout ailleurs.

4.2.3 User Control DualListView

La conception du composant *DualListView* est un exemple concret de développement orienté réutilisabilité. Ce contrôle, utilisé pour la gestion des participants dans les

dépenses et les activités, a simplifié le code des vues concernées et illustre une bonne application des principes de conception modulaire.